Guía 1 de 3 — Listas y tuplas

DPL1046 · Clase 3 · Para repasar en casa, a tu ritmo.

Una **estructura de datos** es una forma de guardar **varios datos juntos** en una sola variable. Hoy conocimos cuatro; en esta guía repasamos las dos que se parecen: la **lista** y la **tupla**.

> Antes de empezar, haz tu rutina de inicio (Guía 1 de la Clase 2): terminal, carpeta del repo, `code .`, activar `.venv` y `git pull`.

---

1. La lista `[ ]`: una lista de compras

Piensa en una lista de compras: pan, leche, huevos. Tiene un **orden** (el pan es el primero) y la puedes **cambiar** (agregar queso, tachar la leche). En Python:

```
compras = ["pan", "leche", "huevos"]
```

Tres reglas para recordar:

1. Se escribe con **corchetes** `[ ]`.
2. Los elementos van **separados por comas**.
3. Cada texto va **entre comillas**.

La posición se cuenta desde 0

Posición	0	1	2
Elemento	pan	leche	huevos

```
compras[0]     # "pan"     (el primero)
compras[2]     # "huevos"
compras[-1]    # "huevos"  (el -1 cuenta desde el final)
len(compras)   # 3         (cuántos hay)
```

> **Truco:** los corchetes **pegados a un nombre** (`compras[0]`) significan “dame el elemento de esta posición”. Los corchetes **al crear** (`[ "pan", ... ]`) significan “esto es una lista”. Son dos usos distintos.

Agregar y recorrer

```
compras.append("queso")        # agrega al final

for producto in compras:       # "para cada producto de la lista..."
    print(" - ", producto)     # ...muestra el producto
```

- `append` = agregar al final. La lista **cambia**: por eso las listas son **modificables**.
- `for ... in ...:` repite lo que está debajo **una vez por elemento**. `producto` es un nombre temporal: vale “pan”, luego “leche”, y así.
- La línea del `print` va **corrida a la derecha** (4 espacios). Esa **sangría** le dice a Python qué se repite.

Lo mismo, con los estados de Olist

```
estados = ["created", "approved", "processing", "shipped", "delivered"]
print("Cuántos:", len(estados))        # 5
print("El primero:", estados[0])       # created
print("El último:", estados[-1])       # delivered
estados.append("canceled")             # ahora hay 6
```

Debes ver (`01_listas.py`):

```
La lista: ['created', 'approved', 'processing', 'shipped', 'delivered']
Cuántos: 5
El primero: created
El último: delivered
Ahora: ['created', 'approved', 'processing', 'shipped', 'delivered', 'canceled']
 - estado: created
 - estado: approved
 ...
 - estado: canceled
```

---

2. La tupla `( )` : una lista “sellada”

Una tupla es como una lista, pero **no se puede modificar** después de crearla. Piensa en tu fecha de nacimiento: `(14, 3, 1990)` . Tiene varias partes, en orden, y no quieres que nadie la cambie por error.

```
fecha = (14, 3, 1990)
fecha[0]          # 14
fecha[0] = 15     # TypeError: una tupla no se puede modificar
```

Se escribe con **paréntesis** `( )` y se lee **igual que una lista**, por posición.

En Olist: la "huella" de una orden

```
ids = orders["order_id"].tolist()                  # columna -> lista
estado_orden = orders["order_status"].tolist()     # columna -> lista
huella = (ids[0], estado_orden[0])                 # tupla: (id, estado)
```

- `orders["order_id"]` es **una columna** de la tabla (como una columna de Excel).
- `.tolist()` la convierte en **lista**.
- `huella` junta el primer id y el primer estado en una tupla. Como el id y el estado de esa orden no deben cambiar, una tupla es una buena elección.

Debes ver (`02_tuplas.py`): una línea con la tupla `('e481f51c…', 'delivered')` , otra con el id y otra con el estado.

---

3. Los errores que vas a ver (y qué significan)

Error	Qué pasó	Cómo se arregla
`IndexError: list index out of range`	Pediste una posición que no existe (`estados[10]` en una lista de 5)	Usa una posición menor que `len(lista)`
`IndentationError`	Falta la **sangría** debajo del `for`	Corre la línea 4 espacios a la derecha
`TypeError: 'tuple' object does not support item assignment`	Intentaste cambiar una tupla	Las tuplas no se cambian: usa una lista si necesitas modificar
`SyntaxError`	Falta una comilla, una coma o un paréntesis	Revisa la línea que marca el error y la anterior
`NameError`	Escribiste un nombre que no existe (por ejemplo, `estado` en vez de `estados`)	Revisa la ortografía del nombre

> Un error **no es un fracaso**: la **última línea** del mensaje dice qué pasó.

---

4. Lista o tupla: ¿cuál uso?

	Lista `[ ]`	Tupla `( )`
Tiene orden	Sí	Sí
Se puede modificar	**Sí**	**No**
Para qué	Datos que cambian: integrantes, columnas a usar, pasos de un proceso	Datos fijos: (id, estado), (día, mes, año), coordenadas

5. Práctica opcional (si quieres reforzar)

No es tarea ni se entrega. Es para ti. Escribe cada ejercicio en un archivo nuevo y ejecútalo.

1. Crea una lista `dias` con los siete días de la semana. Muestra el primero, el último y cuántos hay.
2. Agrega `"feriado"` al final con `append` y recórrela con un `for`.
3. Crea la tupla `punto = (10, 20)` y muestra `punto[1]`. Luego intenta cambiarlo y lee el error.

▼ **Respuestas**

```
dias = ["lunes", "martes", "miércoles", "jueves", "viernes", "sábado",
"domingo"]
print(dias[0], dias[-1], len(dias))     # lunes domingo 7

dias.append("feriado")
for d in dias:
    print(d)

punto = (10, 20)
print(punto[1])      # 20
# punto[1] = 5       # TypeError
```

Vocabulario

Palabra	Qué es
Estructura de datos	Una forma de guardar varios datos juntos
Lista `[ ]`	Datos en orden que se pueden modificar
Tupla `( )`	Datos en orden que **no** se pueden modificar
Posición (índice)	El número de lugar de un elemento; empieza en 0
`append`	Agrega un elemento al final de una lista
`len`	Cuenta cuántos elementos hay
`for`	Repite algo para cada elemento
Sangría	Los espacios a la izquierda que indican qué pertenece al `for`